

10.3-4

Write an $O(n)$ -time procedure that prints out all the keys of an arbitrary rooted tree with n nodes, where the tree is stored using the left-child, right-sibling representation.

★ 10.3-5

Write an $O(n)$ -time nonrecursive procedure that, given an n -node binary tree, prints out the key of each node. Use no more than constant extra space outside of the tree itself and do not modify the tree, even temporarily, during the procedure.

★ 10.3-6

The left-child, right-sibling representation of an arbitrary rooted tree uses three pointers in each node: *left-child*, *right-sibling*, and *parent*. From any node, its parent can be accessed in constant time and all its children can be accessed in time linear in the number of children. Show how to use only two pointers and one boolean value in each node x so that x 's parent or all of x 's children can be accessed in time linear in the number of x 's children.

Problems

10-1 Comparisons among lists

For each of the four types of lists in the following table, what is the asymptotic worst-case running time for each dynamic-set operation listed?

	unsorted, singly linked	sorted, singly linked	unsorted, doubly linked	sorted, doubly linked
SEARCH				
INSERT				
DELETE				
SUCCESSOR				
PREDECESSOR				

MINIMUM				
MAXIMUM				

10-2 Mergeable heaps using linked lists

A *mergeable heap* supports the following operations: MAKE-HEAP (which creates an empty mergeable heap), INSERT, MINIMUM, EXTRACT-MIN, and UNION.¹ Show how to implement mergeable heaps using linked lists in each of the following cases. Try to make each operation as efficient as possible. Analyze the running time of each operation in terms of the size of the dynamic set(s) being operated on.

a. Lists are sorted.

b. Lists are unsorted.

c. Lists are unsorted, and dynamic sets to be merged are disjoint.

10-3 Searching a sorted compact list

We can represent a singly linked list with two arrays, *key* and *next*. Given the index *i* of an element, its value is stored in *key*[*i*], and the index of its successor is given by *next*[*i*], where *next*[*i*] = NIL for the last element. We also need the index *head* of the first element in the list. An *n*-element list stored in this way is *compact* if it is stored only in positions 1 through *n* of the *key* and *next* arrays.

Let's assume that all keys are distinct and that the compact list is also sorted, that is, *key*[*i*] < *key*[*next*[*i*]] for all *i* = 1, 2, ..., *n* such that *next*[*i*] ≠ NIL. Under these assumptions, you will show that the randomized algorithm COMPACT-LIST-SEARCH searches the list for key *k* in $O(\sqrt{n})$ expected time.

COMPACT-LIST-SEARCH(*key*, *next*, *head*, *n*, *k*)

1 *i* = *head*

2 **while** *i* ≠ NIL and *key*[*i*] < *k*

3 *j* = RANDOM(1, *n*)

4 **if** *key*[*i*] < *key*[*j*] and *key*[*j*] ≤ *k*

```

5       $i = j$ 
6      if  $key[i] == k$ 
7          return  $i$ 
8       $i = next[i]$ 
9if  $i == \text{NIL}$  or  $key[i] > k$ 
10     return  $\text{NIL}$ 
11else return  $i$ 

```

If you ignore lines 3–7 of the procedure, you can see that it's an ordinary algorithm for searching a sorted linked list, in which index i points to each position of the list in turn. The search terminates once the index i “falls off” the end of the list or once $key[i] \geq k$. In the latter case, if $key[i] = k$, the procedure has found a key with the value k . If, however, $key[i] > k$, then the search will never find a key with the value k , so that terminating the search was the correct action.

Lines 3–7 attempt to skip ahead to a randomly chosen position j . Such a skip helps if $key[j]$ is larger than $key[i]$ and no larger than k . In such a case, j marks a position in the list that i would reach during an ordinary list search. Because the list is compact, we know that any choice of j between 1 and n indexes some element in the list.

Instead of analyzing the performance of COMPACT-LIST-SEARCH directly, you will analyze a related algorithm, COMPACT-LIST-SEARCH', which executes two separate loops. This algorithm takes an additional parameter t , which specifies an upper bound on the number of iterations of the first loop.

COMPACT-LIST-SEARCH' ($key, next, head, n, k, t$)

```

1  $i = head$ 
2 for  $q = 1$  to  $t$ 
3      $j = \text{RANDOM}(1, n)$ 
4     if  $key[i] < key[j]$  and  $key[j] \leq k$ 
5          $i = j$ 
6         if  $key[i] == k$ 
7             return  $i$ 
8 while  $i \neq \text{NIL}$  and  $key[i] < k$ 

```

```

9   i = next[i]
10  if i == NIL or key[i] > k
11    return NIL
12 else return i

```

To compare the execution of the two algorithms, assume that the sequence of calls of $\text{RANDOM}(1, n)$ yields the same sequence of integers for both algorithms.

- a.** Argue that for any value of t , $\text{COMPACT-LIST-SEARCH}(\text{key}, \text{next}, \text{head}, n, k)$ and $\text{COMPACT-LIST-SEARCH}'(\text{key}, \text{next}, \text{head}, n, k, t)$ return the same result and that the number of iterations of the **while** loop of lines 2–8 in $\text{COMPACT-LIST-SEARCH}$ is at most the total number of iterations of both the **for** and **while** loops in $\text{COMPACT-LIST-SEARCH}'$.

In the call $\text{COMPACT-LIST-SEARCH}'(\text{key}, \text{next}, \text{head}, n, k, t)$, let X_t be the random variable that describes the distance in the linked list (that is, through the chain of *next* pointers) from position i to the desired key k after t iterations of the **for** loop of lines 2–7 have occurred.

- b.** Argue that $\text{COMPACT-LIST-SEARCH}'(\text{key}, \text{next}, \text{head}, n, k, t)$ has an expected running time of $O(t + E[X_t])$.
- c.** Show that $E[X_t] = \sum_{r=1}^n (1-r/n)^t$. (*Hint:* Use equation (C.28) on page 1193.)
- d.** Show that $\sum_{r=0}^{n-1} r^t \leq n^{t+1}/(t+1)$. (*Hint:* Use inequality (A.18) on page 1150.)
- e.** Prove that $E[X_t] \leq n/(t+1)$.
- f.** Show that $\text{COMPACT-LIST-SEARCH}'(\text{key}, \text{next}, \text{head}, n, k, t)$ has an expected running time of $O(t + n/t)$.
- g.** Conclude that $\text{COMPACT-LIST-SEARCH}$ runs in $O(\sqrt{n})$ expected time.

h. Why do we assume that all keys are distinct in COMPACT-LIST-SEARCH? Argue that random skips do not necessarily help asymptotically when the list contains repeated key values.

Chapter notes

Aho, Hopcroft, and Ullman [6] and Knuth [259] are excellent references for elementary data structures. Many other texts cover both basic data structures and their implementation in a particular programming language. Examples of these types of textbooks include Goodrich and Tamassia [196], Main [311], Shaffer [406], and Weiss [452, 453, 454]. The book by Gonnet and Baeza-Yates [193] provides experimental data on the performance of many data-structure operations.

The origin of stacks and queues as data structures in computer science is unclear, since corresponding notions already existed in mathematics and paper-based business practices before the introduction of digital computers. Knuth [259] cites A. M. Turing for the development of stacks for subroutine linkage in 1947.

Pointer-based data structures also seem to be a folk invention. According to Knuth, pointers were apparently used in early computers with drum memories. The A-1 language developed by G. M. Hopper in 1951 represented algebraic formulas as binary trees. Knuth credits the IPL-II language, developed in 1956 by A. Newell, J. C. Shaw, and H. A. Simon, for recognizing the importance and promoting the use of pointers. Their IPL-III language, developed in 1957, included explicit stack operations.

¹ Because we have defined a mergeable heap to support MINIMUM and EXTRACT-MIN, we can also refer to it as a *mergeable min-heap*. Alternatively, if it supports MAXIMUM and EXTRACT-MAX, it is a *mergeable max-heap*.